

```
-- PostgreSQL DDL Script
-- Sistema de Gestão de Farmácia
-- Versão otimizada e refatorada

-- =====
-- Tabela: cliente
-- Dados cadastrais de clientes
-- =====

CREATE TABLE cliente (
    id_cliente SERIAL PRIMARY KEY,
    nome VARCHAR(150) NOT NULL,
    cpf CHAR(11) NOT NULL UNIQUE,
    telefone VARCHAR(15),
    email VARCHAR(100),
    data_cadastro TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    ativo BOOLEAN DEFAULT TRUE,
    CONSTRAINT chk_cpf_length CHECK (LENGTH(cpf) = 11 AND cpf ~ '^\d{11}$'),
    CONSTRAINT chk_email_format CHECK (email IS NULL OR email ~* '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$'),
    CONSTRAINT chk_telefone CHECK (telefone IS NULL OR telefone ~ '^\d{10,11}$')
);

CREATE INDEX idx_cliente_cpf ON cliente(cpf);
CREATE INDEX idx_cliente_nome ON cliente(nome);
CREATE INDEX idx_cliente_email ON cliente(email);

COMMENT ON TABLE cliente IS 'Cadastro de clientes da farmácia';
COMMENT ON COLUMN cliente.cpf IS 'CPF sem formatação (apenas números)';
COMMENT ON COLUMN cliente.telefone IS 'Telefone com DDD sem formatação';

-- =====
-- Tabela: produto
-- Catálogo de produtos
-- =====

CREATE TABLE produto (
    id_produto SERIAL PRIMARY KEY,
    nome VARCHAR(200) NOT NULL,
    codigo_barras VARCHAR(50) UNIQUE,
    categoria VARCHAR(50),
    lote VARCHAR(30),
    data_validade DATE,
    preco_custo NUMERIC(10,2),
    preco_venda NUMERIC(10,2) NOT NULL,
    quantidade_estoque INTEGER DEFAULT 0,
    estoque_minimo INTEGER DEFAULT 0,
    requer_receita BOOLEAN DEFAULT FALSE,
    ativo BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT chk_preco_custo CHECK (preco_custo IS NULL OR preco_custo >= 0),
    CONSTRAINT chk_preco_venda CHECK (preco_venda > 0),
    CONSTRAINT chk_quantidade CHECK (quantidade_estoque >= 0),
    CONSTRAINT chk_estoque_minimo CHECK (estoque_minimo >= 0),
    CONSTRAINT chk_categoria CHECK (categoria IN ('medicamento', 'higiene', 'cosmetico', 'perfumaria', 'suplemento', 'outros'))
);

CREATE INDEX idx_produto_codigo_barras ON produto(codigo_barras);
CREATE INDEX idx_produto_nome ON produto(nome);
CREATE INDEX idx_produto_categoria ON produto(categoria);
CREATE INDEX idx_produto_lote ON produto(lote);
CREATE INDEX idx_produto_estoque_baixo ON produto(quantidade_estoque) WHERE quantidade_estoque <= estoque_minimo;

COMMENT ON TABLE produto IS 'Catálogo de produtos disponíveis';
COMMENT ON COLUMN produto.requer_receita IS 'Indica se o produto requer receita médica';

-- =====
-- Tabela: colaborador
-- Funcionários da farmácia
-- =====

CREATE TABLE colaborador (
    id_colaborador SERIAL PRIMARY KEY,
    nome VARCHAR(150) NOT NULL,
    cpf CHAR(11) NOT NULL UNIQUE,
    matricula VARCHAR(20) UNIQUE,
    cargo VARCHAR(50) NOT NULL,
    crf VARCHAR(20),
    data_admissao DATE DEFAULT CURRENT_DATE,
    data_demissao DATE,
    salario NUMERIC(10,2),
    ativo BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT chk_cpf_colaborador CHECK (LENGTH(cpf) = 11 AND cpf ~ '^\d{11}$'),
    CONSTRAINT chk_cargo CHECK (cargo IN ('farmaceutico', 'atendente', 'gerente', 'estoquista', 'caixa')),
    CONSTRAINT chk_crf_farmaceutico CHECK (
        (cargo = 'farmaceutico' AND crf IS NOT NULL) OR
        (cargo != 'farmaceutico' AND crf IS NULL)
    ),
);
```

```

CONSTRAINT chk_salario CHECK (salario IS NULL OR salario > 0),
CONSTRAINT chk_datas CHECK (data_demissao IS NULL OR data_demissao >= data_admissao)
);

CREATE INDEX idx_colaborador_matricula ON colaborador(matricula);
CREATE INDEX idx_colaborador_cargo ON colaborador(cargo);
CREATE INDEX idx_colaborador_ativo ON colaborador(ativo);

COMMENT ON TABLE colaborador IS 'Cadastro de colaboradores da farmácia';
COMMENT ON COLUMN colaborador.crf IS 'Registro no Conselho Regional de Farmácia (obrigatório para farmacêuticos)';

-- =====
-- Tabela: pedido
-- Pedidos realizados
-- =====
CREATE TABLE pedido (
    id_pedido SERIAL PRIMARY KEY,
    cliente_id INTEGER NOT NULL REFERENCES cliente(id_cliente) ON DELETE RESTRICT,
    colaborador_id INTEGER REFERENCES colaborador(id_colaborador) ON DELETE SET NULL,
    data_pedido TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR(30) DEFAULT 'pendente',
    valor_produtos NUMERIC(10,2) DEFAULT 0,
    valor_desconto NUMERIC(10,2) DEFAULT 0,
    valor_total NUMERIC(10,2) NOT NULL,
    forma_pagamento VARCHAR(30),
    observacao TEXT,
    CONSTRAINT chk_status CHECK (status IN ('pendente', 'processando', 'aguardando_pagamento', 'pago', 'separando', 'pronto', 'entregue',
'cancelado')),
    CONSTRAINT chk_valores CHECK (
        valor_produtos >= 0 AND
        valor_desconto >= 0 AND
        valor_total >= 0 AND
        valor_total = (valor_produtos - valor_desconto)
    ),
    CONSTRAINT chk_forma_pagamento CHECK (forma_pagamento IN ('dinheiro', 'debito', 'credito', 'pix', 'convenio'))
);

CREATE INDEX idx_pedido_cliente ON pedido(cliente_id);
CREATE INDEX idx_pedido_colaborador ON pedido(colaborador_id);
CREATE INDEX idx_pedido_data ON pedido(data_pedido);
CREATE INDEX idx_pedido_status ON pedido(status);

COMMENT ON TABLE pedido IS 'Pedidos de venda realizados';
COMMENT ON COLUMN pedido.colaborador_id IS 'Atendente que realizou o pedido';

-- =====
-- Tabela: item_pedido
-- Itens de cada pedido
-- =====
CREATE TABLE item_pedido (
    id_item_pedido SERIAL PRIMARY KEY,
    pedido_id INTEGER NOT NULL REFERENCES pedido(id_pedido) ON DELETE CASCADE,
    produto_id INTEGER NOT NULL REFERENCES produto(id_produto) ON DELETE RESTRICT,
    quantidade INTEGER NOT NULL,
    preco_unitario NUMERIC(10,2) NOT NULL,
    desconto NUMERIC(10,2) DEFAULT 0,
    subtotal NUMERIC(10,2) NOT NULL,
    CONSTRAINT chk_quantidade CHECK (quantidade > 0),
    CONSTRAINT chk_preco_unitario CHECK (preco_unitario > 0),
    CONSTRAINT chk_desconto CHECK (desconto >= 0),
    CONSTRAINT chk_subtotal CHECK (
        subtotal >= 0 AND
        subtotal = ((quantidade * preco_unitario) - desconto)
    ),
    CONSTRAINT uk_item_pedido UNIQUE (pedido_id, produto_id)
);

CREATE INDEX idx_item_pedido_pedido ON item_pedido(pedido_id);
CREATE INDEX idx_item_pedido_produto ON item_pedido(produto_id);

COMMENT ON TABLE item_pedido IS 'Itens individuais de cada pedido';

-- =====
-- Tabela: localizacao_estoque
-- Localização física dos produtos no estoque
-- =====
CREATE TABLE localizacao_estoque (
    id_localizacao SERIAL PRIMARY KEY,
    produto_id INTEGER NOT NULL REFERENCES produto(id_produto) ON DELETE CASCADE,
    corredor VARCHAR(10) NOT NULL,
    prateleira VARCHAR(10) NOT NULL,
    secao VARCHAR(10),
    posicao VARCHAR(10),
    quantidade INTEGER DEFAULT 0,
    CONSTRAINT chk_quantidade_localizacao CHECK (quantidade >= 0),
    CONSTRAINT uk_localizacao UNIQUE (corredor, prateleira, secao, posicao)
);

```

```
CREATE INDEX idx_localizacao_produto ON localizacao_estoque(produto_id);
CREATE INDEX idx_localizacao_endereco ON localizacao_estoque(corredor, prateleira, secao);

COMMENT ON TABLE localizacao_estoque IS 'Localização física dos produtos no estoque';
COMMENT ON COLUMN localizacao_estoque.quantidade IS 'Quantidade do produto nesta localização específica';

-- =====
-- Tabela: validacao_produto
-- Registro de validações de produtos
-- =====

CREATE TABLE validacao_produto (
    id_validacao SERIAL PRIMARY KEY,
    produto_id INTEGER NOT NULL REFERENCES produto(id_produto) ON DELETE CASCADE,
    farmaceutico_id INTEGER NOT NULL REFERENCES colaborador(id_colaborador) ON DELETE RESTRICT,
    data_validacao TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    tipo_validacao VARCHAR(50) NOT NULL,
    status VARCHAR(30) NOT NULL,
    observacao TEXT,
    CONSTRAINT chk_tipo_validacao CHECK (tipo_validacao IN ('entrada', 'saida', 'inventario', 'validade', 'qualidade', 'receita')),
    CONSTRAINT chk_status_validacao CHECK (status IN ('aprovado', 'reprovado', 'pendente', 'em_analise'))
);

CREATE INDEX idx_validacao_produto ON validacao_produto(produto_id);
CREATE INDEX idx_validacao_farmaceutico ON validacao_produto(farmaceutico_id);
CREATE INDEX idx_validacao_data ON validacao_produto(data_validacao);
CREATE INDEX idx_validacao_status ON validacao_produto(status);

COMMENT ON TABLE validacao_produto IS 'Registro de validações realizadas por farmacêuticos';

-- =====
-- Triggers para atualização automática
-- =====

-- Função para atualizar timestamp
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = CURRENT_TIMESTAMP;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Trigger para produto
CREATE TRIGGER update_produto_updated_at
BEFORE UPDATE ON produto
FOR EACH ROW
EXECUTE FUNCTION update_updated_at_column();

-- Função para atualizar valor total do pedido
CREATE OR REPLACE FUNCTION update_pedido_valor_total()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE pedido
    SET valor_produtos = (
        SELECT COALESCE(SUM(subtotal), 0)
        FROM item_pedido
        WHERE pedido_id = COALESCE(NEW.pedido_id, OLD.pedido_id)
    ),
    valor_total = (
        SELECT COALESCE(SUM(subtotal), 0) - COALESCE(valor_desconto, 0)
        FROM item_pedido
        WHERE pedido_id = COALESCE(NEW.pedido_id, OLD.pedido_id)
    )
    WHERE id_pedido = COALESCE(NEW.pedido_id, OLD.pedido_id);

    RETURN COALESCE(NEW, OLD);
END;
$$ LANGUAGE plpgsql;

-- Trigger para recalcular valor do pedido
CREATE TRIGGER update_pedido_total_on_item_insert
AFTER INSERT ON item_pedido
FOR EACH ROW
EXECUTE FUNCTION update_pedido_valor_total();

CREATE TRIGGER update_pedido_total_on_item_update
AFTER UPDATE ON item_pedido
FOR EACH ROW
EXECUTE FUNCTION update_pedido_valor_total();

CREATE TRIGGER update_pedido_total_on_item_delete
AFTER DELETE ON item_pedido
FOR EACH ROW
EXECUTE FUNCTION update_pedido_valor_total();

-- =====
```

```
-- Views úteis
-- =====

-- View: Produtos com estoque baixo
CREATE OR REPLACE VIEW v_produtos_estoque_baixo AS
SELECT
    p.id_produto,
    p.nome,
    p.codigo_barras,
    p.categoria,
    p.quantidade_estoque,
    p.estoque_minimo,
    (p.estoque_minimo - p.quantidade_estoque) AS quantidade_repor
FROM produto p
WHERE p.quantidade_estoque <= p.estoque_minimo
    AND p.ativo = TRUE
ORDER BY (p.estoque_minimo - p.quantidade_estoque) DESC;

COMMENT ON VIEW v_produtos_estoque_baixo IS 'Produtos que precisam de reposição no estoque';

-- View: Produtos próximos ao vencimento
CREATE OR REPLACE VIEW v_produtos_vencimento_proximo AS
SELECT
    p.id_produto,
    p.nome,
    p.lote,
    p.data_validade,
    (p.data_validade - CURRENT_DATE) AS dias_para_vencer,
    p.quantidade_estoque
FROM produto p
WHERE p.data_validade IS NOT NULL
    AND p.data_validade <= CURRENT_DATE + INTERVAL '90 days'
    AND p.data_validade >= CURRENT_DATE
    AND p.ativo = TRUE
ORDER BY p.data_validade;

COMMENT ON VIEW v_produtos_vencimento_proximo IS 'Produtos com vencimento nos próximos 90 dias';

-- View: Resumo de pedidos
CREATE OR REPLACE VIEW v_resumo_pedidos AS
SELECT
    p.id_pedido,
    p.data_pedido,
    c.nome AS cliente,
    c.cpf AS cliente_cpf,
    col.nome AS atendente,
    p.status,
    p.valor_total,
    p.forma_pagamento,
    COUNT(ip.id_item_pedido) AS quantidade_itens,
    SUM(ip.quantidade) AS quantidade_produtos
FROM pedido p
INNER JOIN cliente c ON c.id_cliente = p.cliente_id
LEFT JOIN colaborador col ON col.id_colaborador = p.colaborador_id
LEFT JOIN item_pedido ip ON ip.pedido_id = p.id_pedido
GROUP BY p.id_pedido, p.data_pedido, c.nome, c.cpf, col.nome, p.status, p.valor_total, p.forma_pagamento
ORDER BY p.data_pedido DESC;

COMMENT ON VIEW v_resumo_pedidos IS 'Resumo completo dos pedidos com informações do cliente e atendente';

-- View: Validações pendentes
CREATE OR REPLACE VIEW v_validacoes_pendentes AS
SELECT
    v.id_validacao,
    p.nome AS produto,
    p.codigo_barras,
    p.lote,
    v.tipo_validacao,
    v.status,
    v.data_validacao,
    f.nome AS farmaceutico,
    vobservacao
FROM validacao_produto v
INNER JOIN produto p ON p.id_produto = v.produto_id
INNER JOIN colaborador f ON f.id_colaborador = v.farmaceutico_id
WHERE v.status IN ('pendente', 'em_analise')
ORDER BY v.data_validacao;

COMMENT ON VIEW v_validacoes_pendentes IS 'Validações que aguardam conclusão';

-- View: Performance de colaboradores (vendas)
CREATE OR REPLACE VIEW v_performance_colaboradores AS
SELECT
    col.id_colaborador,
    col.nome,
    col.cargo,
    COUNT(DISTINCT p.id_pedido) AS total_pedidos,
```

```
SUM(p.valor_total) AS valor_total_vendas,
AVG(p.valor_total) AS ticket_medio,
COUNT(DISTINCT DATE(p.data_pedido)) AS dias_com_vendas
FROM colaborador col
LEFT JOIN pedido p ON p.colaborador_id = col.id_colaborador
AND p.status IN ('pago', 'pronto', 'entregue')
WHERE col.ativo = TRUE
GROUP BY col.id_colaborador, col.nome, col.cargo
ORDER BY valor_total_vendas DESC NULLS LAST;

COMMENT ON VIEW v_performance_colaboradores IS 'Performance de vendas dos colaboradores';
```